fractol Runtime Line-by-Line (شروع اجرا تا پایان)

این نسخه دقیقاً همان چیزی است که خواستی:

- نه فقط توضیح کلی
- نه فقط توضیح فایل‌به‌فایل
- «بلکه «وقتی برنامه اجرا می‌شود، از کدام خط شروع می‌کند و هر خط چه می‌کند

کدهای مرجع:

- `main.c`
- `utils.c`
- `events.c`
- `render.c`
- `fractol.h`

---

0) اجرای برنامه از کجا شروع می‌شود؟

وقتی می‌زنی:

```
./fractol julia -0.8 0.156
```

CPU وارد `main` می‌شود:

- `main.c:79`

---

1) `main` — مسیر اجرای واقعی خط‌به‌خط

کد:

```
int main(int ac, char **av)
{
    t_data  data;

    init_data(&data);
    if (!parse_args(ac, av, &data))
        return (1);
    if (setup_graphics(&data))
        return (1);
    render(&data);
    mlx_key_hook(data.win, key_hook, &data);
    mlx_mouse_hook(data.win, mouse_hook, &data);
    mlx_hook(data.win, 17, 0, close_hook, &data);
    mlx_loop(data.mlx);
```

```
    return (0);
}
```

`main.c:81` → `t_data data;`

- چه می‌کند: یک object از نوع `t_data` روی stack می‌سازد.
- چرا لازم است: کل برنامه را نگه می‌دارد state (window, image, zoom, set...).
- اگر نبود: نمی‌توانستی یکپارچه داشته باشی state.

`main.c:83` → `init_data(&data);`

- چه می‌کند: مقدار اولیه امن برای همه فیلدها می‌گذارد.
- بعدش برنامه می‌رود داخل `init_data` در `main.c:29`.

---

2) `init_data` — خط‌به‌خط (همین لحظه اجرا می‌شود)

کد:

```
void    init_data(t_data *data)
{
    data->mlx = NULL;
    data->win = NULL;
    data->img = NULL;
    data->addr = NULL;
    data->bits_per_pixel = 0;
    data->line_length = 0;
    data->endian = 0;
    data->zoom = 1.0;
    data->shift_x = 0.0;
    data->shift_y = 0.0;
    data->julia_x = -0.744;
    data->julia_y = 0.148;
    data->color_shift = 0;
    data->set = MANDELBROT;
}
```

`main.c:31` → `data->mlx = NULL;`

- چه می‌کند: pointer context را خالی می‌گذارد.
- چرا لازم است: قبل از `mlx_init` معتبر نیست.

`main.c:32` → `data->win = NULL;`

- پنجره هنوز ساخته نشده pointer.

`main.c:33` → `data->img = NULL;`

- تصویر هنوز ساخته نشده pointer.

`main.c:34` → `data->addr = NULL;`

- آدرس buffer هنوز معلوم نیست.

`main.c:35` → `bits_per_pixel = 0;`

- metadata اولیه امن.

`main.c:36` → `line_length = 0;`

- metadata اولیه امن.

`main.c:37` → `endian = 0;`

- metadata اولیه امن.

`main.c:38` → `zoom = 1.0;`

- zoom (بدون بزرگنمایی) پایه.

`main.c:39` → `shift_x = 0.0;`

- جابجایی افقی اولیه.

`main.c:40` → `shift_y = 0.0;`

- جابجایی عمودی اولیه.

`main.c:41` → `julia_x = -0.744;`

- `c` پیش‌فرض Julia (real).

`main.c:42` → `julia_y = 0.148;`

- `c` پیش‌فرض Julia (imag).

`main.c:43` → `color_shift = 0;`

- پالت رنگ از حالت پایه شروع می‌شود.

`main.c:44` → `set = MANDELBROT;`

- set پیش‌فرض (اگر parse بعداً تغییر ندهد).

خروج از `init_data` و برگشت به `main`.

---

3) ورودی parse و `main` برگشت به

`main.c:84` → `if (!parse_args(ac, av, &data))`

- ورودی: چه می‌کند command line را بررسی می‌کند.
- `!` یعنی اگر `parse_args` صفر برگرداند (fail)، وارد `if` شو.

- حالا برنامه می‌رود داخل `parse_args` در `utils.c:85`.

---

4) `parse_args` — خط‌به‌خط با مسیر واقعی همین اجرا

فرض اجرای واقعی ما:

```
./fractol julia -0.8 0.156
```

پس:

- `ac = 4`
- `av[1] = "julia"`
- `av[2] = "-0.8"`
- `av[3] = "0.156"`

کد:

```
int parse_args(int ac, char **av, t_data *data)
{
    if (ac == 2 && streq(av[1], "mandelbrot"))
        data->set = MANDELBROT;
    else if (ac == 2 && (streq(av[1], "burningship")
            || streq(av[1], "burning_ship")))
        data->set = BURNING_SHIP;
    else if (ac == 2 && streq(av[1], "julia"))
        data->set = JULIA;
    else if (ac == 4 && streq(av[1], "julia")
        && ft_atof_strict(av[2], &data->julia_x)
        && ft_atof_strict(av[3], &data->julia_y))
        data->set = JULIA;
    else
        return (usage());
    set_default_view(data);
    return (1);
}
```

`utils.c:87` → `if (ac == 2 && streq(..."mandelbrot"))`

- چون `ac=4`، بخش `ac == 2` **false** می‌شود.
- به خاطر short-circuit، `streq` اصلاً صدا زده نمی‌شود.

`utils.c:89` → `else if (ac == 2 && (...burningship...))`

- باز هم `ac == 2` false.
- هیچ `streq` اجرا نمی‌شود.

`utils.c:92` → `else if (ac == 2 && streq(..."julia"))`

- باز false.

`utils.c:94` → `else if (ac == 4 && streq(av[1], "julia") ... )`

- `ac == 4` true.
- حالا `streq(av[1], "julia")` اجرا می‌شود.

`streq` اجرا می‌شود (`utils.c:15`)

کد:

```
static int  streq(char *a, char *b)
{
    while (*a && *b && *a == *b)
    {
        ++a;
        ++b;
    }
    return (*a == '\0' && *b == '\0');
}
```

- مقایسه می‌شود `"julia"` با `"julia"` کاراکتر به کاراکتر.
- در انتها هر دو به `0\` می‌رسند → `return 1`.

برگشت به شرط `utils.c:94`.

`utils.c:95` → `ft_atof_strict(av[2], &data->julia_x)`

- `av[2] = "-0.8"`.
- اگر parse موفق شود، مقدار در `data->julia_x` ریخته می‌شود.

اجرای `ft_atof_strict` (`utils.c:56`) برای `"0.8-"`

1. `if (!s || !*s || !out)` → false (ورودی معتبر است).
2. `res = 0.0; frac = 0.1; sign = 1; started = 0;`
3. `if (*s == '-' || *s == '+')` true چون `'-'`.
4. `sign = -2 * (*s++ == '-') + 1;` → `sign = -1`, و `s` یک کاراکتر جلو می‌رود.
5. `if (*s >= '0' && *s <= '9') started = 1;` `0'` برای true.
6. `parse_digits(..., frac_mode=0)` بخش صحیح را parse (`0`) می‌کند.
7. `if (*s == '.') ++s;` از ممیز رد می‌شود.
8. `if (*s >= '0' && *s <= '9') started = 1;` `8'` برای true.
9. `parse_digits(..., frac_mode=1)` بخش اعشاری را parse (`0.8`) می‌کند.
10. `if (*s || !started)` false (چیزی اضافه نمانده).
11. `*out = res * sign` → `-0.8`.
12. `return 1`.

`utils.c:96` → `ft_atof_strict(av[3], &data->julia_y)`

- `av[3] = "0.156"` parse می‌شود.

- خروجی: `data->julia_y = 0.156`.

`utils.c:97` → `data->set = JULIA;`

- set انتخابی نهایی: Julia.

`utils.c:100` → `set_default_view(data);`

- بعد از parse، view مناسب set تنظیم می‌شود.
- حالا برنامه می‌رود `events.c:22`.

---

5) `set_default_view` — خط‌به‌خط در مسیر فعلی

کد:

```
void    set_default_view(t_data *data)
{
    data->zoom = 1.0;
    data->color_shift = 0;
    if (data->set == JULIA)
    {
        data->shift_x = 0.0;
        data->shift_y = 0.0;
        return ;
    }
    data->shift_x = -0.5;
    data->shift_y = 0.0;
    if (data->set == BURNING_SHIP)
        data->shift_y = -0.5;
}
```

اجرای واقعی برای set = JULIA

1. `zoom = 1.0`
2. `color_shift = 0`
3. شرط `data->set == JULIA` true.
4. `shift_x = 0.0`
5. `shift_y = 0.0`
6. `return` → خطوط Mandelbrot/BurningShip اجرا نمی‌شوند.

برگشت به `parse_args`.

`utils.c:101` → `return (1);`

- parse موفق.
- برگشت به `main`.

---

6) گرافیک setup و `main` برگشت به

`main.c:84` شرط `if (!parse_args...)`

- چون parse `1` وارد، پس `!1 = 0`، برگرداند `return (1)` نمی‌شود.

`main.c:86` → `if (setup_graphics(&data))`

- می‌رود داخل `setup_graphics` (`main.c:60`).

---

7) `setup_graphics` — خط‌به‌خط

کد:

```
int setup_graphics(t_data *data)
{
    data->mlx = mlx_init();
    if (!data->mlx)
        return (put_err("Error: mlx_init failed\n"));
    data->win = mlx_new_window(data->mlx, WIDTH, HEIGHT, "fract-ol");
    if (!data->win)
        return (destroy_data(data), put_err("Error: mlx_new_window
failed\n"));
    data->img = mlx_new_image(data->mlx, WIDTH, HEIGHT);
    if (!data->img)
        return (destroy_data(data), put_err("Error: mlx_new_image
failed\n"));
    data->addr = mlx_get_data_addr(data->img, &data->bits_per_pixel,
            &data->line_length, &data->endian);
    if (!data->addr)
        return (destroy_data(data),
            put_err("Error: mlx_get_data_addr failed\n"));
    return (0);
}
```

1. `mlx_init()` context می‌سازد.
2. اگر fail شود، error چاپ و خروج.
3. `mlx_new_window(..., WIDTH, HEIGHT, "fract-ol")` پنجره می‌سازد.
4. اگر fail شود، cleanup و خروج.
5. `mlx_new_image(...)` buffer تصویر می‌سازد.
6. اگر fail شود، cleanup و خروج.
7. `mlx_get_data_addr(...)` آدرس raw pixel memory و metadata می‌گیرد.
8. اگر fail شود، cleanup و خروج.
9. `return 0` یعنی setup موفق.

برگشت به `main`.

`main.c:86` شرط

- چون setup `0` داده، وارد `return (1)` نمی‌شود.

---

8) اولین رندر (`main.c:88`)

`main.c:88` → `render(&data);`

- برنامه وارد `render` در `render.c:94` می‌شود.

---

9) `render` — خط‌به‌خط (واقعا چه اتفاقی می‌افتد)

کد:

```
void    render(t_data *data)
{
    t_render    r;
    int         *row;

    r.sx = 4.0 / (WIDTH * data->zoom);
    r.sy = 4.0 / (HEIGHT * data->zoom);
    r.start_x = (-WIDTH / 2.0) * r.sx + data->shift_x;
    r.start_y = (-HEIGHT / 2.0) * r.sy + data->shift_y;
    r.y = 0;
    while (r.y < HEIGHT)
    {
        row = (int *)(data->addr + r.y * data->line_length);
        r.x = 0;
        while (r.x < WIDTH)
        {
            r.cx = r.start_x + r.x * r.sx;
            r.cy = r.start_y + r.y * r.sy;
            row[r.x] = get_palette_color(get_point_iter(data, r.cx, r.cy),
                    data->color_shift);
            ++r.x;
        }
        ++r.y;
    }
    mlx_put_image_to_window(data->mlx, data->win, data->img, 0, 0);
}
```

خطوط setup mapping

1. `r.sx` اندازه هر pixel روی محور x در صفحه مختلط.
2. `r.sy` اندازه هر pixel روی محور y.
3. `r.start_x` مختصات مختلط پیکسل x=0.
4. `r.start_y` مختصات مختلط پیکسل y=0.
5. `r.y = 0` شروع از سطر اول.

حلقه y (`while (r.y < HEIGHT)`)

- برای هر سطر تصویر یک بار اجرا می‌شود (800 بار).

`row = (int *)(data->addr + r.y * data->line_length);`

- آدرس ابتدای سطر `r.y` در buffer.

حلقه x (`while (r.x < WIDTH)`)

- برای هر پیکسل سطر اجرا می‌شود (800 بار).

`r.cx = ...` و `r.cy = ...`

- پیکسل `(x,y)` به نقطه مختلط `(cx, cy)` تبدیل می‌شود.

`get_point_iter(...)`

- تعداد iteration تا escape را برای همین نقطه می‌دهد.

`get_palette_color(...)`

- iteration را به رنگ تبدیل می‌کند (با `color_shift`).

`row[r.x] = ...`

- رنگ پیکسل داخل buffer نوشته می‌شود.

پایان

- بعد از تمام pixelها، `mlx_put_image_to_window` تصویر را روی پنجره می‌گذارد.

---

10) داخل `get_point_iter` — ریاضی دقیق هر پیکسل

کد:

```
int get_point_iter(t_data *data, double cx, double cy)
{
    double  z[2];
    double  tmp;
    int     i;

    init_iter(data, &cx, &cy, z);
    i = 0;
    while (i < MAX_ITER && (z[0] * z[0] + z[1] * z[1]) < 4.0)
    {
        if (data->set == BURNING_SHIP)
        {
            z[0] = fabs(z[0]);
            z[1] = fabs(z[1]);
        }
        tmp = z[0] * z[0] - z[1] * z[1] + cx;
        z[1] = 2.0 * z[0] * z[1] + cy;
        z[0] = tmp;
        ++i;
```

```
    }
    return (i);
}
```

`init_iter(data, &cx, &cy, z)`

- اگر `JULIA`:
  - `z0 = pixel`
  - `c = (julia_x, julia_y)`
- اگر `MANDELBROT/BURNING_SHIP`:
  - `z0 = 0`
  - `c = pixel`

حلقه while

- شرط 1: `i < MAX_ITER` (حداکثر 100 تکرار)
- شرط 2: `|z|^2 < 4.0` یعنی هنوز escape نکرده.

branch Burning Ship

- قبل از squaring، مختصات `z` قدرمطلق می‌شود.

فرمول update

- `tmp = x^2 - y^2 + cx`
- `y = 2xy + cy`
- `x = tmp`
- `i++`

`return (i)`

- اگر نزدیک set باشد `i` بالا.
- اگر سریع فرار کند `i` پایین.

---

11) داخل `get_palette_color` و `get_color`

`get_palette_color`

1. اولین بار `palette` ساخته می‌شود (`ready == 0`).
2. `0` برای `..MAX_ITER` از `get_color` رنگ می‌سازد.
3. اگر `iter == MAX_ITER` رنگ داخل set (`palette[MAX_ITER]` = سیاه).
4. در غیر این‌صورت با `shift` index جابه‌جا می‌شود.

`get_color`

1. `iter == MAX_ITER` -> `0x000000`.
2. `t = iter / MAX_ITER`.
3. فرمول polynomial برای RGB.
4. برگرداندن رنگ packed.

---

12) برگشت به `main` بعد از اولین render

`main.c:89` → `mlx_key_hook(data.win, key_hook, &data);`

- callback کیبورد ثبت می‌شود.

`main.c:90` → `mlx_mouse_hook(data.win, mouse_hook, &data);`

- callback mouse ثبت می‌شود.

`main.c:91` → `mlx_hook(data.win, 17, 0, close_hook, &data);`

- callback بستن پنجره (X) ثبت می‌شود.

`main.c:92` → `mlx_loop(data.mlx);`

- برنامه وارد event loop می‌شود و منتظر event می‌ماند.
- از اینجا به بعد، flow با eventها جلو می‌رود.

---

13) مسیر Runtime هنگام Eventها

13.0) از کجا برنامه می‌فهمد کدام دکمه را زدی؟

این همان سؤال اصلی evaluatorها هم هست.

جریان واقعی از سیستم‌عامل تا کد شما:

1. شما روی کیبورد یا ماوس عمل می‌کنی.
2. در Linux/X11، event توسط X server تولید می‌شود.
3. MiniLibX داخل `mlx_loop` این eventها را از صف event می‌خواند.
4. چون در `main` callbackهای ثبت کرده‌ای:
   - `mlx_key_hook(data.win, key_hook, &data);`
   - `mlx_mouse_hook(data.win, mouse_hook, &data);`
   - `mlx_hook(data.win, 17, 0, close_hook, &data);` پس MLX callbackها را صدا مناسب می‌زند.
5. MLX callbackها را خودش پر می‌کند:
   - `key_hook(int keycode, t_data *data)` -> `keycode` از event کیبورد می‌آید.
   - `mouse_hook(int button, int x, int y, t_data *data)` -> `button/x/y` از event ماوس می‌آیند.
6. `data` همان pointer موقع register کردن callbackهای داده‌ای (`&data` در `main`).

شما با عدد دکمه/کلید صدا زده، callback می‌کند، MLX dispatch می‌دهد، OS/X11 event پس برنامه «حدس» نمی‌زند؛ می‌شود.

keycodeها از کجا آمده‌اند؟

در `fractol.h` این هدر را include کرده‌ای:

```
#include <X11/keysym.h>
```

وقتی می‌نویسی `key_hook` را تعریف می‌کند. پس در `XK_Left`, `XK_Right`, `XK_Escape`, `XK_c` این هدر ثابت‌هایی مثل:

```
if (keycode == XK_Left) ...
```

مقایسه می‌کنی Left را با مقدار استاندارد کلید X11 یعنی عدد دریافتی از.

از کجا آمده‌اند؟ mouse های button

در Linux/X11 معمولاً:

1. `button == 1` -> کلیک چپ
2. `button == 2` -> کلیک وسط
3. `button == 3` -> کلیک راست
4. `button == 4` -> wheel up
5. `button == 5` -> wheel down

استفاده کرده‌ای zoom تو فقط `4` و `5` را برای.

event `17` چیست؟

در:

```
mlx_hook(data.win, 17, 0, close_hook, &data);
```

یعنی `17` عدد event بستن پنجره (روی Linux/X11).
وقتی روی `X` پنجره کلیک می‌کنی، MLX تابع `close_hook` را صدا می‌زند.

مثال واقعی 1 (کلید Left)

فرض کاربر کلید Left را می‌زند:

1. X11 یک key event می‌فرستد.
2. MLX داخل loop این event را می‌گیرد.
3. `key_hook(keycode, &data)` صدا زده می‌شود.
4. `keycode == XK_Left` true می‌شود.
5. `data->shift_x -= MOVE_STEP / data->zoom`
6. `render(data)` اجرا می‌شود -> تصویر با view جدید redraw می‌شود.

مثال واقعی 2 (روی نقطه خاص Wheel Up)

فرض کاربر wheel up را روی `(x=620, y=300)` می‌زند:

1. MLX صدا می‌زند: `mouse_hook(4, 620, 300, &data)`
2. چون `button == 4`:

    - `zoom_at_point(data, 620, 300, ZOOM_FACTOR)` اجرا می‌شود.
3. `zoom_at_point`:
    - نقطه مختلط زیر ماوس را قبل zoom ذخیره می‌کند.
    - zoom را افزایش می‌دهد.
    - shift زیر را تنظیم می‌کند تا همان نقطه cursor بماند.
4. `render(data)` -> تصویر جدید.

3 مثال واقعی (بستن پنجره با X)

1. پنجره کلیک می‌کند X کاربر روی.
2. event شماره 17 تولید می‌شود.
3. MLX -> `close_hook(&data)`.
4. `destroy_data(data)` همه resourceها را آزاد می‌کند.
5. `exit(0)` برنامه تمیز خارج می‌شود.

13.1) Mouse wheel (zoom)

وقتی wheel بالا/پایین می‌چرخد:

- وارد `mouse_hook` (`events.c:56`) می‌شود.

خط‌به‌خط `mouse_hook`

1. اگر `(x,y)` بیرون پنجره باشد: `return 0`.
2. `button == 4` -> `zoom_at_point(..., ZOOM_FACTOR)`.
3. `button == 5` -> `zoom_at_point(..., 1.0 / ZOOM_FACTOR)`.
4. در غیر این‌صورت ignore.
5. `render(data)` دوباره صدا می‌شود.

خط‌به‌خط `zoom_at_point`

1. scale فعلی را حساب می‌کند.
2. مختصات مختلط زیر mouse را قبل zoom ذخیره می‌کند.
3. `data->zoom *= factor`.
4. scale جدید را حساب می‌کند.
5. `shift_x/shift_y` زیر را طوری تنظیم می‌کند که همان نقطه mouse بماند.

13.2) Keyboard

وقتی کلید می‌زنی:

- وارد `key_hook` (`events.c:70`) می‌شود.

خط‌به‌خط `key_hook`

1. `XK_Escape` -> `close_hook(data)`.
2. `XK_Left` -> `shift_x -= MOVE_STEP / zoom`.
3. `XK_Right` -> `shift_x += MOVE_STEP / zoom`.
4. `XK_Up` -> `shift_y -= MOVE_STEP / zoom`.

5. `XK_Down` -> `shift_y += MOVE_STEP / zoom`.
6. `XK_c/XK_C` -> `color_shift += 2`.
7. `XK_v/XK_V` -> `color_shift -= 2`.
8. `XK_r/XK_R` -> `set_default_view(data)`.
9. اگر key پشتیبانی نشود: `return 0`.
10. در مسیر معتبر: `render(data)` برای اعمال تغییر.

13.3) Close window / ESC

`close_hook` (`events.c:15`)

1. `destroy_data(data)` cleanup منابع MLX.
2. `exit(0)` خروج.

`destroy_data` (`main.c:47`)

1. اگر image و mlx معتبرند: `mlx_destroy_image`.
2. اگر window و mlx معتبرند: `mlx_destroy_window`.
3. اگر mlx معتبر است:
   - `mlx_destroy_display`
   - `free(data->mlx)`

---

14) مسیر خطاها (Runtime واقعی)

14.1) ورودی غلط

- `main.c:84` -> `parse_args` صفر برمی‌گرداند.
- `main` در `main.c:85` با `return 1` خارج می‌شود.
- قبلش `usage()` در `utils.c:25` متن راهنما را چاپ کرده.

14.2) خطای گرافیکی

- اگر `mlx_init`/`mlx_new_window`/`mlx_new_image`/`mlx_get_data_addr` fail شوند:
- `setup_graphics` مقدار `1` برمی‌گرداند.
- `main` در `main.c:87` خارج می‌شود.

---

15) خلاصه خیلی کوتاه Flow کامل

1. Start: `main.c:79`
2. Init state: `init_data`
3. Parse CLI: `parse_args`
4. Set default camera: `set_default_view`
5. Init MLX: `setup_graphics`
6. First draw: `render`
7. Register callbacks
8. Enter `mlx_loop`
9. هر event -> update state -> `render`

10. خروج با ESC یا close button -> `destroy_data` -> `exit(0)`

---

16) Trace کامل Runtime برای همه حالت‌های اجرا

در این بخش، دقیقاً مسیر اجرا برای همه ورودی‌های اصلی را می‌بینی.

16.1) حالت `mandelbrot`

فرمان:

```
./fractol mandelbrot
```

Flow دقیق:

1. `main.c:79` شروع.
2. `init_data` اجرا می‌شود.
3. `parse_args`:
   - شرط `ac == 2 && streq(av[1], "mandelbrot")` true.
   - `data->set = MANDELBROT`.
   - `set_default_view`:
     - `zoom = 1.0`
     - `color_shift = 0`
     - چون `JULIA` نیست:
       - `shift_x = -0.5`
       - `shift_y = 0.0`
4. `setup_graphics` موفق.
5. `render`:
   - در `init_iter` چون set = `MANDELBROT`:
     - `z0 = 0`
     - `c = pixel`
6. حلقه event و تعامل کاربر.

16.2) حالت `julia` بدون پارامتر

فرمان:

```
./fractol julia
```

Flow دقیق:

1. `main` و `init_data`.
2. `parse_args`:
   - شرط `ac == 2 && streq(..., "julia")` true.
   - `data->set = JULIA`.
   - `set_default_view`:

        - `zoom = 1.0`
        - `color_shift = 0`
        - `shift_x = 0.0`
        - `shift_y = 0.0`
3. مقادیر Julia همان defaultهای `init_data` می‌مانند:
    - `julia_x = -0.744`
    - `julia_y = 0.148`
4. در `render/get_point_iter`:
    - `z0 = pixel`
    - `c = (-0.744, 0.148)`

16.3) حالت `julia` با پارامتر

فرمان:

```
./fractol julia -0.8 0.156
```

Flow:

1. `parse_args` می‌رسد به شرط `ac == 4 && streq(..., "julia") && ...`.
2. `ft_atof_strict("-0.8", &julia_x)` -> success.
3. `ft_atof_strict("0.156", &julia_y)` -> success.
4. `set = JULIA`, بعد `set_default_view`.
5. حالا Julia با `c = (-0.8, 0.156)` رسم می‌شود.

16.4) حالت `burningship`

فرمان:

```
./fractol burningship
```

Flow:

1. `parse_args` شرط دوم را true می‌کند.
2. `set = BURNING_SHIP`.
3. `set_default_view`:
    - `shift_x = -0.5`
    - `shift_y = -0.5`
4. در `get_point_iter`:
    - قبل از update:
        - `z[0] = fabs(z[0])`
        - `z[1] = fabs(z[1])`

16.5) حالت ورودی غلط

فرمان:

```
./fractol abc
```

Flow:

1. هیچ شرطی در `parse_args` match نمی‌شود.
2. `return usage();` (مقدار 0) اجرا می‌شود.
3. `main`: `if (!parse_args(...)) return (1);`
4. برنامه با کد خطا خارج می‌شود.

---

17) Line-by-Line تکمیلی برای helperهایی که کوتاه بودند

17.1) `put_err` در `main.c`

کد:

```
static int  put_err(char *msg)
{
    int len;
    int w;

    len = 0;
    while (msg[len])
        ++len;
    w = write(2, msg, len);
    if (w < 0)
        return (1);
    return (1);
}
```

توضیح:

1. `static` یعنی فقط داخل همین فایل دیده می‌شود.
2. `len` طول message.
3. `w` نتیجه `write`.
4. loop طول رشته را می‌شمارد.
5. `write(2, ...)` چاپ روی `stderr`.
6. حتی اگر write fail شود، همچنان error-code (`1`) می‌دهد.

17.2) `usage` در `utils.c`

1. `static char msg[] = ...` پیام ثابت راهنما.
2. `write(1, msg, sizeof(msg)-1)` چاپ کل پیام.
3. برای `0` برگشت fail در parse.

17.3) `parse_digits` در `utils.c`

این تابع یک parser کوچک است که «فقط رقم‌های پشت‌سرهم» را می‌خواند.

ورودی‌ها:

1. `char **s`:
   - آدرس pointer فعلی روی رشته.
   - تابع خودش pointer را جلو می‌برد.
2. `double *res`:
   - accumulator عدد نهایی.
3. `double *frac`:
   - فقط در حالت اعشاری استفاده می‌شود (`0.1`, `0.01`, `0.001`, ...).
4. `int frac_mode`:
   - `0` = parse بخش صحیح (integer part)
   - `1` = parse بخش اعشاری (fractional part)

منطق:

1. تا وقتی `**s` یک digit باشد، loop ادامه دارد.
2. اگر `frac_mode == 0`:
   - `*res = *res * 10 + digit`
   - یعنی مثل ماشین‌حساب بخش صحیح را می‌سازد.
3. اگر `frac_mode == 1`:
   - `*res += digit * (*frac)`
   - بعد `*frac *= 0.1`
   - یعنی هر رقم اعشاری وزن کوچک‌تری می‌گیرد.
4. در هر دور `++(*s)` اجرا می‌شود تا parser روی کاراکتر بعدی برود.

مثال 1 (بخش صحیح)

فرض:

- رشته: `"abc123"`
- `s` روی `'1'`
- `res = 0`
- `frac_mode = 0`

مراحل:

1. digit=`1` -> `res=1`
2. digit=`2` -> `res=12`
3. digit=`3` -> `res=123`
4. `'a'` دیگر digit نیست -> loop stop

خروجی:

- `res = 123`
- `s` حالا روی `'a'` ایستاده.

مثال 2 (بخش اعشاری)

فرض:

- `156"` :رشته`x"` (از dot یعنی قسمت بعد)
- `s 1'` روی `'`
- `res = 0`
- `frac = 0.1`
- `frac_mode = 1`

مراحل:

1. digit=`1` -> `res += 1*0.1 = 0.1`, سپس `frac=0.01`
2. digit=`5` -> `res += 5*0.01 = 0.05` => `res=0.15`, سپس `frac=0.001`
3. digit=`6` -> `res += 6*0.001 = 0.006` => `res=0.156`
4. `'x'` digit نیست -> stop

خروجی:

- `res = 0.156`
- `s` روی `'x'`.

3 مثال (اتصال به `ft_atof_strict`)

`0.156-"` برای ورودی`"`:

1. `ft_atof_strict` اول sign را می‌خواند (`-`).
2. `parse_digits(..., frac_mode=0)` عدد `0` را می‌سازد.
3. رد می‌شود `.` از.
4. `parse_digits(..., frac_mode=1)` مقدار `0.156` را اضافه می‌کند.
5. در آخر sign `0.156-` <- اعمال می‌شود.

پس این تابع خودش کل عدد را parse «بخش» فقط digit و نمی‌کند؛ را می‌خواند «pointer های فعلی» را جلو می‌برد.

17.4) `init_iter` در `render.c`

1. اگر set = `JULIA`:
   - `z0 = pixel` (در `z[0], z[1]`)
   - `c = julia_x + i*julia_y` (با overwrite روی `cx,cy`)
2. در غیر این‌صورت:
   - `z0 = 0`
   - `c = pixel` (همان `cx,cy` اصلی)

این دقیقاً فرق ریاضی Julia با Mandelbrot/BurningShip است.

---

18) مثال عددی واقعی (Iteration-by-Iteration)

18.1) Mandelbrot برای نقطه `c = 0 + 0i`

شروع:

- `z0 = 0`

iteration:

1. `z1 = z0^2 + c = 0 + 0 = 0`
2. `z2 = 0`
3. ... 0 همیشه

نتیجه:

- escape نمی‌کند
- `iter = MAX_ITER`
- رنگ: سیاه

18.2) Mandelbrot برای نقطه `c = 2 + 0i`

شروع:

- `z0 = 0`

iteration:

1. `z1 = 0^2 + 2 = 2`
2. `|z1|^2 = 4` → شرط while (باید باشد `4.0 >`) دیگر برقرار نیست

نتیجه:

- escape خیلی سریع
- iteration کم
- رنگ روشن/خارج از set

18.3) Julia با نمونه `c = -0.8 + 0.156i`

فرض یک پیکسل که map به شده:

- `z0 = 0.2 + 0.3i`

iteration 1:

- `x' = x^2 - y^2 + cx = 0.04 - 0.09 - 0.8 = -0.85`
- `y' = 2xy + cy = 2*0.2*0.3 + 0.156 = 0.276`
- `z1 = -0.85 + 0.276i`

iterationتا های بعد همین‌طور ادامه پیدا می‌کند escape یا رسیدن به `MAX_ITER`.

---

19) Mapping (خیلی مهم) پیکسل به صفحه مختلط

در `render`:

```
r.sx = 4.0 / (WIDTH * zoom);
r.sy = 4.0 / (HEIGHT * zoom);
```

```
r.start_x = (-WIDTH / 2.0) * r.sx + shift_x;
r.start_y = (-HEIGHT / 2.0) * r.sy + shift_y;
cx = r.start_x + x * r.sx;
cy = r.start_y + y * r.sy;
```

معنی:

1. بازه پایه تقریبی محور: طول 4 واحد.
2. وقتی `zoom` زیاد می‌شود، `sx/sy` کوچک‌تر می‌شوند (یعنی zoom in).
3. `shift_x/shift_y` پنجره دید را جابه‌جا می‌کنند.
4. هر پیکسل `(x,y)` یک نقطه `(cx,cy)` در صفحه مختلط می‌شود.

مثال:

- `WIDTH = 800`, `zoom = 1`
- `sx = 4/800 = 0.005`
- اگر `shift_x = -0.5`، پیکسل وسط (`x=400`) می‌شود:
  - `cx = start_x + 400*0.005 = -0.5`

---

20) چرا `|z|^2 < 4.0`؟

در ریاضی fractal (برای این خانواده):

- اگر `|z| > 2` شود، مسیر قطعاً به بی‌نهایت می‌رود.
- برای سرعت، به‌جای `sqrt(x^2+y^2) > 2` از `x^2 + y^2 > 4` استفاده می‌شود.

مزیت:

- بدون `sqrt`
- سریع‌تر در هر پیکسل.

---

21) جزئیات Zoom حول Mouse (با مثال)

هدف: نقطه زیر cursor بعد از zoom هم همان نقطه بماند.

فرآیند:

1. نقطه مختلط زیر mouse قبل از zoom را حساب کن (`mouse_re`, `mouse_im`).
2. zoom را تغییر بده.
3. scale جدید را حساب کن.
4. shift جدید را طوری تنظیم کن که:
   - `mouse_re/mouse_im` دوباره به همان pixel cursor map شود.

اگر این کار را نکنی:

- zoom همیشه حول مرکز انجام می‌شود.
- حس کنترل ضعیف‌تر می‌شود.

---

22) چرا حرکت با `MOVE_STEP / zoom`؟

اگر حرکت ثابت باشد:

- در zoom بالا، یک step خیلی زیاد دیده می‌شود.

با تقسیم بر zoom:

- هر چه zoom بیشتر، حرکت ظریف‌تر.
- کنترل بهتر در deep zoom.

---

23) حافظه و Resource Lifetime

23.1) چه چیزهایی allocate می‌شوند؟

1. `mlx_init` -> MLX context.
2. `mlx_new_window` -> window resource.
3. `mlx_new_image` -> image resource.

23.2) چه چیزهایی آزاد می‌شوند؟

در `destroy_data`:

1. image
2. window
3. display + free(mlx)

23.3) چه چیزهایی stack هستند؟

- `t_data data;` در `main` (stack).
- `t_render r;` در `render` (stack).

23.4) static lifetime

- `palette` و `ready` در `get_palette_color`.

---

24) پیچیدگی زمانی و کارایی

هر frame:

- حلقه روی `WIDTH * HEIGHT` پیکسل.
- برای هر پیکسل تا `MAX_ITER` iteration.

تقریب:

- `O(WIDTH * HEIGHT * MAX_ITER)`
- با مقادیر فعلی: `64,000,000 = 100 * 800 * 800` گام (حداکثر).

نکته performance:

- precompute palette باعث می‌شود رنگ‌دهی سریع‌تر شود.

---

25) چک‌لیست ارزیابی (Mandatory + Bonus)

Mandatory:

1. `./fractol mandelbrot` کار می‌کند.
2. `./fractol julia` کار می‌کند.
3. `./fractol julia <x> <y>` کار می‌کند.
4. wheel zoom کار می‌کند.
5. ESC خروج تمیز.
6. close button خروج تمیز.
7. usage برای ورودی غلط چاپ می‌شود.
8. رنگ‌بندی depth وجود دارد.

Bonus:

1. `burningship` فعال است.
2. zoom بر اساس mouse position.
3. حرکت با arrow keys.
4. color shift با `C`/`V`.

---

26) سوال‌های رایج evaluator + جواب کوتاه

1. چرا `MAX_ITER = 100`؟

- تعادل speed/quality. بیشترش جزئیات بیشتر می‌دهد ولی کندتر می‌شود.

2. چرا از `|z|^2 < 4` استفاده کردی؟

- معادل `|z| < 2` است ولی بدون `sqrt` و سریع‌تر.

3. فرق Mandelbrot و Julia در کد تو چیست؟

- در `init_iter`:
  Mandelbrot/BurningShip: `z0=0, c=pixel`
  Julia: `z0=pixel, c=constant`

4. رنگ‌ها چطور تولید می‌شوند؟

- `iter` نرمال می‌شود به `t` و با فرمول polynomial به RGB تبدیل می‌شود.

5. چرا `set_default_view` بعد از parse صدا زده می‌شود؟

- چون framing مناسب هر fractal متفاوت است و بعد از مشخص شدن set باید تنظیم شود.

---

27) اشکال‌زدایی سریع (اگر چیزی کار نکرد)

1. اگر برنامه باز نشد:

- X11/Display را چک کن.
- `mlx_init` ممکن است fail شود.

2. اگر Julia با پارامتر کار نکرد:

- ورودی عددی strict `12` است. `a` یا `.` reject می‌شود.

3. اگر zoom عجیب شد:

- `zoom` صفر یا منفی نمی‌شود (factor‌ها کنترل‌شده‌اند).

4. اگر رنگ‌ها تغییر نکرد:

- کلیدهای `C` یا `V` بزن، `color_shift` تغییر می‌کند و render مجدد باید رخ دهد.

---

28) نقشه ذهنی نهایی (Mental Model)

`fractol` = یک loop گرافیکی event-driven:

1. State initialize.
2. Input parse.
3. Graphics setup.
4. Draw all pixels using complex math.
5. Wait for events.
6. On event: update state -> redraw.
7. On close: cleanup resources -> exit.

اگر این 7 قدم را بفهمی، کل پروژه را واقعاً فهمیده‌ای.

---

29) ریاضی پایه‌ای که باید 100٪ بلد باشی

برای این پروژه، هسته ریاضی این است:

- عدد مختلط: `z = x + i y`
- در کد:
  - `x` همان `z[0]`
  - `y` همان `z[1]`

29.1) مشتق مستقیم فرمول `z^2`

اگر:

- `z = x + i y`

آنگاه:

- `z^2 = (x + i y)^2`
- `z^2 = x^2 + 2ixy + i^2 y^2`
- چون `i^2 = -1`:
- `z^2 = (x^2 - y^2) + i(2xy)`

پس وقتی می‌گوییم:

- `z_{n+1} = z_n^2 + c`

یعنی:

- `x_{n+1} = x_n^2 - y_n^2 + c_x`
- `y_{n+1} = 2 x_n y_n + c_y`

این دقیقاً همان کد تو است در `render.c`:

```
tmp = z[0] * z[0] - z[1] * z[1] + cx;
z[1] = 2.0 * z[0] * z[1] + cy;
z[0] = tmp;
```

29.2) چرا از `tmp` استفاده شده؟

اگر مستقیم بنویسی:

- اول `z[0]` را آپدیت کنی
- بعد برای `z[1]` از `z[0]` استفاده کنی

فرمول غلط می‌شود، چون `z[1]` باید از **x قدیمی** استفاده کند، نه x جدید.

پس:

- `tmp` real جدید را موقت نگه می‌دارد
- imaginary با x,y قدیمی حساب می‌شود
- در آخر `z[0] = tmp`

این نکته معمولاً evaluator می‌پرسد.

---

30) شرط Escape چرا `|z|^2 < 4.0` است؟

در کد شرط loop:

```
(z[0] * z[0] + z[1] * z[1]) < 4.0
```

یعنی:

- `|z|^2 < 4`
- معادل `|z| < 2`

30.1) چرا شعاع 2؟

اگر قدرمطلق از 2 رد شود، دنباله به سمت بی‌نهایت می‌رود (escape). برای این خانواده (`z^2 + c`)

30.2) چرا مربع قدرمطلق و نه خود قدرمطلق؟

- `|z| = sqrt(x^2 + y^2)`
- گرفتن `sqrt` برای هر iteration کند است.
- پس مستقیم `x^2 + y^2` مقایسه می‌کنیم با `4` را.

این optimization استاندارد fractol است.

---

31) تفاوت دقیق سه Fractal در کد تو

31.1) Mandelbrot

- `z0 = 0`
- `c = pixel`

در کد:

- داخل `init_iter` چون `set != JULIA`:
  - `z[0] = 0.0`
  - `z[1] = 0.0`
- و `cx, cy` همان مختصات پیکسل می‌مانند.

31.2) Julia

- `z0 = pixel`
- `c = constant (julia_x, julia_y)`

در کد (`init_iter`):

- `z[0] = *cx`
- `z[1] = *cy`
- `*cx = data->julia_x`
- `*cy = data->julia_y`

یعنی:

- `z` می‌رود داخل pixel
- `c` می‌رود داخل Julia ثابت

31.3) Burning Ship

مثل Mandelbrot است، اما قبل از مربع کردن:

- `x = |x|`
- `y = |y|`

در کد:

```
if (data->set == BURNING_SHIP)
{
    z[0] = fabs(z[0]);
```

```
    z[1] = fabs(z[1]);
}
```

این تغییر کوچک، شکل fractal را کاملاً عوض می‌کند.

---

32) Mapping صفحه مختلط ↔ کامل پیکسل

در `render`:

```
r.sx = 4.0 / (WIDTH * zoom);
r.sy = 4.0 / (HEIGHT * zoom);
r.start_x = (-WIDTH / 2.0) * r.sx + shift_x;
r.start_y = (-HEIGHT / 2.0) * r.sy + shift_y;
cx = start_x + x * sx;
cy = start_y + y * sy;
```

32.1) معنی هندسی

- صفحه نمایش 800x800 است.
- ما یک «پنجره» روی صفحه مختلط می‌گیریم.
- `sx/sy` می‌گویند هر پیکسل چند واحد مختلط است.

32.2) نقش `zoom`

- `zoom` زیاد شود -> `sx/sy` کوچک می‌شوند <- جزئیات بیشتر (zoom in).
- `zoom` کم شود -> `sx/sy` بزرگ می‌شوند <- نمای بازتر (zoom out).

32.3) نقش `shift_x/shift_y`

- کل پنجره دید را روی صفحه مختلط جابه‌جا می‌کنند.
- Arrow keys دقیقاً همین‌ها را تغییر می‌دهند.

32.4) معکوس (دانستن برای دفاع) mapping

اگر evaluator «بپرسد: «مختصات مختلط معلوم، پیکسلش کجاست؟

از:

- `cx = start_x + x*sx` -> `x = (cx - start_x)/sx`
- `cy = start_y + y*sy` -> `y = (cy - start_y)/sy`

---

33) اثبات ریاضی Zoom around Mouse در کد تو

تابع: `zoom_at_point`

ایده:

- قبل از zoom، نقطه مختلط زیر mouse را ذخیره می‌کنیم: `mouse_re`, `mouse_im`.

- بعد zoom عوض می‌شود.
- بعد shift که mouse را طوری تنظیم می‌کنیم دوباره همان نقطه را نشان بدهد.

فرمول کد:

```
mouse_re = (x - WIDTH/2.0)*scale_x + shift_x;
zoom *= factor;
shift_x = mouse_re - (x - WIDTH/2.0)*new_scale_x;
```

چرا درست است؟

باید این برقرار باشد update بعد از:

- `mouse_re = (x - WIDTH/2)*new_scale_x + new_shift_x`

پس:

- `new_shift_x = mouse_re - (x - WIDTH/2)*new_scale_x`

دقیقاً همان کد.

نتیجه:

- zoom حول cursor مرکز نه حول، می‌شود انجام.

---

34) رنگ‌ها: ریاضی و رفتار بصری

34.1) `get_color(iter)`

- اگر `iter == MAX_ITER` -> سیاه (داخل set).
- در غیر این‌صورت:
  - `t = iter / MAX_ITER`
  - RGB با polynomial نرم محاسبه می‌شوند.

چرا polynomial؟

- transition ساده نسبت به رنگ‌پله‌ای نرم‌تر می‌دهد.

34.2) `get_palette_color(iter, shift)`

- palette یکبار ساخته می‌شود (`static` + `ready`).
- هر بار فقط index جابجا می‌شود:
  - `i = (iter + shift) % MAX_ITER`

34.3) چرا modulo با correction منفی؟

ممکن است `%` برای منفی، نتیجه منفی بدهد. پس C در:

```
if (i < 0)
    i += MAX_ITER;
```

تا index همیشه داخل بازه معتبر باشد.

---

35) Parsing و Validation: دقیق و دفاع‌پذیر

35.1) ورودی‌های معتبر

1. `./fractol mandelbrot`
2. `./fractol burningship`
3. `./fractol burning_ship`
4. `./fractol julia`
5. `./fractol julia <x> <y>`

35.2) ورودی‌های نامعتبر (نمونه)

1. `./fractol`
2. `./fractol abc`
3. `./fractol julia 1`
4. `./fractol julia x y`
5. `./fractol julia 1.2.3 0`

35.3) `ft_atof_strict` چه چیزی را reject می‌کند؟

- کاراکتر اضافی: `12a`
- بدون رقم: `.` یا `+` یا `-`
- رشته خالی

35.4) چرا strict parser خوب است؟

- evaluator می‌بیند input validation واقعی داری.
- رفتار deterministic می‌ماند.

---

36) Function Contracts (خیلی مهم برای دفاع)

این بخش مثل قرارداد رسمی هر تابع است.

36.1) `init_data(t_data *data)`

- ورودی: pointer معتبر به `t_data`
- خروجی: ندارد (`void`)
- اثر: کل state را مقداردهی اولیه می‌کند
- بعد از خروج invariant:
  - pointerها `NULL`
  - zoom = 1

- defaults Julia آماده

36.2) `parse_args(int ac, char **av, t_data *data)`

- ورودی: خط فرمان args
- خروجی:
  - `1` = موفق parse
  - `0` = ناموفق parse (usage چاپ شده)
- اثر: `data->set` و احتمالاً `julia_x/y` تنظیم می‌شوند

36.3) `setup_graphics(t_data *data)`

- خروجی:
  - `0` = موفق
  - `1` = خطا
- اثر: `mlx`, `win`, `img`, `addr` و metadata مقدار می‌گیرند

36.4) `render(t_data *data)`

- خروجی: ندارد
- اثر: کل buffer فعلی state تصویر با وضعیت روی window پر می‌شود و گذاشته می‌شود

36.5) `mouse_hook(...)`, `key_hook(...)`

- ورودی: event
- خروجی: `0`
- اثر: state تغییر می‌کند و در صورت معتبر event، `render` می‌شود

36.6) `destroy_data(t_data *data)`

- خروجی: ندارد
- اثر: تمام resourceهای MLX که ساخته شده‌اند آزاد می‌شوند

---

37) سناریوی کامل Event Timeline (مثال واقعی)

سناریو:

1. اجرا: `./fractol mandelbrot`
2. تصویر اول می‌آید.
3. کاربر wheel up می‌زند:
   - `mouse_hook` -> `zoom_at_point` -> `render`
4. کاربر `Right` می‌زند:
   - `key_hook` -> `shift_x += MOVE_STEP/zoom` -> `render`
5. کاربر `C` می‌زند:
   - `color_shift += 2` -> `render`
6. کاربر `R` می‌زند:
   - `set_default_view` -> `render`
7. کاربر ESC:

- `close_hook` -> `destroy_data` -> `exit(0)`

اگر evaluator بگوید «Flow همین، «بگو واقعی را timeline است عالی جواب.

---

38) سوالات سخت evaluator + جواب پیشنهادی

Q1) چرا `set_default_view` داخل `parse_args` صدا زده‌ای؟

A:

- چون framing هر پیش‌فرض set فرق دارد.
- تنظیم می‌شود set مشخص شد، دوربین اولیه متناسب با همان set بعد از اینکه.

Q2) چرا از `static palette` استفاده کردی؟

A:

- برای جلوگیری از محاسبه مجدد رنگ‌ها در هر پیکسل.
- ها سریع‌تر می‌شوندframe فقط یک‌بار ساخته می‌شود و.

Q3) اگر `mlx_new_window` fail کند چه می‌شود؟

A:

- تابع `setup_graphics` خطا را detect می‌کند، cleanup برمی‌گرداند `1` و، می‌کند.
- `main` با خطا خارج می‌شود.

Q4) چرا خروج با `return (1)` در parse/setup خطا درست است؟

A:

- چون non-zero status نشان‌دهنده failure در Unix است.

Q5) تفاوت `button == 4` و `button == 5` چیست؟

A:

- روی Linux/X11: 4 = wheel up، 5 = wheel down.

Q6) چرا داخل حرکت از `MOVE_STEP / zoom` استفاده کردی؟

A:

- تا حرکت screen-space در و ثابت بماند تقریباً zoom شود کنترل دقیق‌تر بالا.

Q7) چرا `julia` بدون پارامتر هم قبول می‌کنی؟

A:

- کار می‌کند Julia های معنی‌دارdefault بهتر است و با usability.
- همچنان `julia <x> <y>` هم پشتیبانی می‌شود.

Q8) اگر evaluator بگوید یک کلید جدید اضافه کن چه می‌کنی؟

A:

- در `key_hook` یک branch جدید اضافه می‌کنم.
- state لازم را تغییر می‌دهم.
- `render(data)` صدا می‌زنم.

---

39) چک‌لیست نهایی دفاع (قبل از evaluation)

1. `norminette *.c *.h` پاس.
2. `make`, `make bonus`, `make fclean` پاس.
3. ورودی غلط usage می‌دهد.
4. 3 هر set اجرا می‌شوند.
5. zoom/move/color/close درست کار می‌کنند.
6. می‌توانی شفاهی این را بگویی:
    - state model (`t_data`)
    - parse flow
    - render mapping
    - iteration formula
    - event-driven redraw
    - cleanup path

---

40) «پاسخ خیلی کوتاه 60 ثانیه‌ای برای «پروژه‌ات چطور کار می‌کند؟

`fractol` من یک برنامه event-driven با MiniLibX است. در `main` اول state را init می‌کنم، args را parse تا می‌کنم را set می‌کنم و یک setup بعد گرافیک را (`MANDELBROT`, `JULIA`, `BURNING_SHIP`) تعیین کنم. در render می‌کنم و render اولیه می‌زنم. با شرط `MAX_ITER` تا `z = z^2 + c` تکرار می‌کنم و هر پیکسل را به یک نقطه در صفحه مختلط می‌کنم، با فرمول map escape `|z|^2 >= 4` تشخیص می‌دهم داخل/خارج set تعداد iteration است. بعد با event را به رنگ تبدیل می‌کنم. برای خروج تمیز داریم ESC/X و color shift برای C/V، view برای حرکت arrow، zoom around mouse برای scroll، hooks. cleanup کامل در `destroy_data` انجام می‌شود.

---

41) اگر evaluator تغییر کوچک بخواهد، چطور سریع انجام بدهی؟

مثال 1: «MAX_ITER را runtime قابل تغییر کن»

- یک فیلد جدید به `t_data` اضافه کن (مثلاً `max_iter`).
- در `init_data` مقدار اولیه بده.
- در `key_hook` تغییر بده با `-/+`.
- در `get_point_iter` به‌جای `MAX_ITER` از `data->max_iter` استفاده کن.

مثال 2: «کلید عوض کردن در لحظه set»

- در `key_hook` روی key خاص:
    - `data->set` را cycle کن.
    - `set_default_view(data)`

    - `render(data)`

«سرعت» مثال 3: «را تغییر بده zoom»

- `ZOOM_FACTOR` را عوض کن یا runtime variable کن.

---

42) جمع‌بندی نهایی خیلی مهم

اگر این 5 مورد را بلد باشی، تقریباً هر سوال evaluator را جواب می‌دهی:

1. **State**: چه چیزهایی داخل `t_data` است و چرا.
2. **Parse**: چطور input معتبر/نامعتبر تشخیص می‌دهی.
3. **Math**: فرمول `z^2 + c` و شرط escape `|z|^2 < 4`.
4. **Render**: mapping پیکسل به صفحه مختلط + loopها.
5. **Events/Cleanup**: چطور interaction و خروج تمیز انجام می‌شود.

تو الآن ساختار فنی لازم را داری؛ با این سند می‌توانی دفاع را خیلی مطمئن انجام بدهی.

---

43) `main.c` کامل، تابع‌به‌تابع (one by one)

این بخش دقیقاً برای همین سؤال است:
«کامل چیست؟ flow دقیقاً چه کار می‌کند و `main.c` هر تابع در»

43.1) `put_err(char *msg)`

کد:

```
static int  put_err(char *msg)
{
    int len;
    int w;

    len = 0;
    while (msg[len])
        ++len;
    w = write(2, msg, len);
    if (w < 0)
        return (1);
    return (1);
}
```

کارکرد:

1. طول رشته error را دستی حساب می‌کند (`len`).
2. با `write(2, ...)` پیام را روی `stderr` چاپ می‌کند.
3. همیشه `1` برمی‌گرداند (یعنی error status).

چرا `static`؟

1. قابل استفاده است `main.c` یعنی فقط داخل.

نکته دفاع:

1. عملاً رفتار خروجی را تغییر نمی‌دهد (چون در هر حالت `1` برمی‌گردانی)، اما نشان `if (w < 0) return (1);` هم نادیده گرفته نشده `write` می‌دهد خطای.

مثال:

1. اگر `mlx_init` fail شود:
   - `put_err("Error: mlx_init failed\n")`
   - پیام چاپ می‌شود
   - کد خطا `1` برمی‌گردد.

43.2) `init_data(t_data *data)`

کد state اولیه را امن می‌کند:

1. همه pointerها `NULL` می‌شوند (`mlx`, `win`, `img`, `addr`).
2. metadata تصویر صفر می‌شود (`bits_per_pixel`, `line_length`, `endian`).
3. view اولیه:
   - `zoom = 1.0`
   - `shift_x = 0.0`
   - `shift_y = 0.0`
4. Julia default constant:
   - `julia_x = -0.744`
   - `julia_y = 0.148`
5. رنگ:
   - `color_shift = 0`
6. set پیش‌فرض:
   - `MANDELBROT`

چرا مهم است؟

1. اگر setup نصفه‌نیمه fail کند، `destroy_data` فقط چیزهایی را که واقعاً ساخته شده‌اند آزاد می‌کند.
2. از undefined behavior جلوگیری می‌شود.

43.3) `destroy_data(t_data *data)`

ترتیب cleanup:

1. اگر `img` و `mlx` معتبرند -> `mlx_destroy_image`.
2. اگر `win` و `mlx` معتبرند -> `mlx_destroy_window`.
3. اگر `mlx` معتبر است:
   - `mlx_destroy_display`
   - `free(data->mlx)`

چرا این ترتیب خوب است؟

1. child resourceها (image/window) قبل از context کلی آزاد می‌شوند.
2. شرط‌ها جلوی crash روی pointerهای `NULL` را می‌گیرند.

کِی صدا زده می‌شود؟

1. هنگام exit طبیعی (`ESC` یا close window).
2. هنگام خطای setup در `setup_graphics`.

43.4) `setup_graphics(t_data *data)`

این تابع مرحله‌به‌مرحله گرافیک را آماده می‌کند:

1. `mlx_init()` -> ساخت MLX context.
2. `mlx_new_window(...)` -> ساخت window.
3. `mlx_new_image(...)` -> ساخت image buffer.
4. `mlx_get_data_addr(...)` -> گرفتن pointer raw pixel memory + metadata.

اگر در هر مرحله fail شود:

1. resourceهای قبلی cleanup می‌شوند (`destroy_data(data)`).
2. پیام خطا چاپ می‌شود (`put_err(...)`).
3. تابع `1` برمی‌گرداند.

نکته syntax:

1. این خط:
   - `return (destroy_data(data), put_err("..."));`
2. با comma operator کار می‌کند:
   - اول cleanup
   - بعد `put_err` اجرا
   - مقدار نهایی return همان مقدار `put_err` (`1` یعنی) است.

43.5) `main(int ac, char **av)`

Flow اصلی برنامه:

1. `t_data data;` -> ساخت state روی stack.
2. `init_data(&data);` -> مقداردهی اولیه امن.
3. `parse_args(ac, av, &data)`:
   - اگر fail -> `return 1`.
4. `setup_graphics(&data)`:
   - اگر fail -> `return 1`.
5. `render(&data);` -> اولین frame.
6. register callbackها:
   - keyboard: `mlx_key_hook`
   - mouse: `mlx_mouse_hook`
   - close window event 17: `mlx_hook`
7. `mlx_loop(data.mlx);`:
   - ورود به loop بی‌نهایت event-driven.
   - از اینجا به بعد OS event می‌دهد و callbackهای شما اجرا می‌شوند.
8. `return 0;`:
   - تمام شود loop در حالت واقعی فقط وقتی (عملاً بعد exit).

43.6) سه سناریوی واقعی با همین `main.c`

سناریو A: اجرای صحیح

فرمان:

```
./fractol julia -0.8 0.156
```

اتفاق:

1. parse موفق.
2. setup گرافیک موفق.
3. render اولیه.
4. loop شروع.
5. کاربر event می‌دهد -> callback -> render مجدد.

سناریو B: ورودی غلط

فرمان:

```
./fractol bad
```

اتفاق:

1. `parse_args` fail.
2. `main` مستقیم `return 1`.
3. هیچ window ساخته نمی‌شود.

سناریو C: خطای گرافیک

اتفاق:

1. parse موفق.
2. یکی از توابع MLX fail (مثلاً `mlx_new_image`).
3. `setup_graphics` cleanup می‌کند + error می‌دهد `1` + چاپ می‌کند.
4. `main` با `return 1` خارج می‌شود.

---

44) پوشش 100٪ همه توابع (20/20) — line-by-line compact

این بخش برای این است که دقیقاً بگویی:

1. همه توابع پروژه پوشش داده شده‌اند.
2. هر تابع خط‌به‌خط توضیح دارد (compact و دفاع‌پذیر).

44.1) `main.c`

1) `put_err`

1. `len = 0` مقدار اولیه.
2. `while (msg[len])` طول message را می‌شمارد.
3. `write(2, msg, len)` روی `stderr` چاپ می‌کند.
4. `if (w < 0)` خطای write را چک می‌کند.
5. در هر حالت `1` برمی‌گرداند (error code).

2) `init_data`

1. pointerها `NULL`.
2. metadata تصویر (`bpp`, `line_length`, `endian`) صفر.
3. camera default (`zoom=1`, `shift_x/y=0`).
4. Julia default (`-0.744`, `0.148`).
5. `color_shift=0`.
6. `set=MANDELBROT`.

3) `destroy_data`

1. اگر `img && mlx` -> `mlx_destroy_image`.
2. اگر `win && mlx` -> `mlx_destroy_window`.
3. اگر `mlx`:
   - `mlx_destroy_display`
   - `free(mlx)`

4) `setup_graphics`

1. `mlx_init` context.
2. اگر fail -> `put_err`.
3. `mlx_new_window`.
4. اگر fail -> `destroy_data` + `put_err`.
5. `mlx_new_image`.
6. اگر fail -> `destroy_data` + `put_err`.
7. `mlx_get_data_addr` برای buffer + metadata.
8. اگر fail -> `destroy_data` + `put_err`.
9. success -> `return 0`.

5) `main`

1. `t_data data` ساخته می‌شود.
2. `init_data`.
3. `parse_args` fail -> `return 1`.
4. `setup_graphics` fail -> `return 1`.
5. `render` اولیه.
6. hook کیبورد.
7. hook ماوس.
8. hook close (`event 17`).
9. `mlx_loop`.
10. `return 0` (بعد از خروج loop).

44.2) `events.c`

6) `close_hook`

1. `destroy_data`.
2. `exit(0)`.
3. `return 0` عملاً unreachable.

7) `set_default_view`

1. `zoom=1`, `color_shift=0`.
2. اگر `JULIA`:
   - `shift_x=0`
   - `shift_y=0`
   - `return`
3. در غیر Julia:
   - `shift_x=-0.5`
   - `shift_y=0`
4. اگر `BURNING_SHIP` -> `shift_y=-0.5`.

8) `zoom_at_point`

1. scale فعلی (`scale_x/scale_y`) محاسبه می‌شود.
2. نقطه مختلط زیر mouse قبل zoom ذخیره می‌شود (`mouse_re/im`).
3. `zoom *= factor`.
4. scale جدید محاسبه می‌شود.
5. `shift_x/shift_y` تنظیم می‌شود تا anchor زیر mouse ثابت بماند.

9) `mouse_hook`

1. اگر `(x,y)` بیرون window -> `return 0`.
2. `button==4` -> zoom in.
3. `button==5` -> zoom out.
4. سایر buttonها -> ignore.
5. `render` و `return 0`.

10) `key_hook`

1. `ESC` -> `close_hook`.
2. `Left` -> `shift_x -= MOVE_STEP/zoom`.
3. `Right` -> `shift_x += MOVE_STEP/zoom`.
4. `Up` -> `shift_y -= MOVE_STEP/zoom`.
5. `Down` -> `shift_y += MOVE_STEP/zoom`.
6. `C/c` -> `color_shift += 2`.
7. `V/v` -> `color_shift -= 2`.
8. `R/r` -> `set_default_view`.
9. کلید ناشناخته -> `return 0`.
10. در مسیر معتبر -> `render`, `return 0`.

44.3) `utils.c`

11) `streq`

1. تا وقتی هر دو رشته حرف دارند و برابرند جلو می‌رود.
2. در پایان اگر هر دو همزمان `'0\'` باشند -> برابر (`1`).
3. وگرنه -> `0`.

12) `usage`

1. پیام ثابت usage تعریف شده.
2. `write(1, msg, sizeof(msg)-1)` چاپ می‌کند.
3. حتی در خطای write هم خروجی parse-fail (`0`) حفظ می‌شود.

13) `parse_digits`

1. تا وقتی char فعلی digit است loop.
2. در `frac_mode=0`:
   - `res = res * 10 + digit`
3. در `frac_mode=1`:
   - `res += digit * frac`
   - `frac *= 0.1`
4. رشته جلو می‌رود هر دور pointer.

14) `ft_atof_strict`

1. guard برای `NULL`/empty.
2. init متغیرها (`res`, `frac`, `sign`, `started`).
3. sign (`+/-`) parse می‌شود.
4. digit قبل dot اگر باشد -> `started=1`.
5. `parse_digits(..., 0)` بخش صحیح.
6. اگر dot باشد از آن عبور می‌کند.
7. digit بعد dot اگر باشد -> `started=1`.
8. `parse_digits(..., 1)` بخش اعشاری.
9. اگر کاراکتر اضافه مانده یا هیچ digit نبود -> fail.
10. `*out = res * sign`.
11. success -> `return 1`.

15) `parse_args`

1. حالت `mandelbrot` با `ac==2`.
2. حالت `burningship` یا `burning_ship` با `ac==2`.
3. حالت `julia` بدون پارامتر با `ac==2`.
4. حالت `julia x y` با `ac==4` + دو parse موفق.
5. اگر هیچکدام نبود -> `usage()` و fail.
6. در مسیر معتبر -> `set_default_view`.
7. success -> `return 1`.

44.4) `render.c`

16) `get_color`

1. اگر `iter == MAX_ITER` -> سیاه.
2. `t = iter / MAX_ITER`.
3. `r/g/b` با فرمول smooth polynomial.
4. packing رنگ با `(r << 16 | g << 8 | b)`.

17) `get_palette_color`

1. `static palette` و `ready`.
2. اگر `ready==0`:
   - `0` از `..MAX_ITER` رنگ‌ها با `get_color` ساخته می‌شوند.
   - `ready=1`.
3. اگر `iter==MAX_ITER` -> رنگ داخل set.
4. index = `(iter + shift) % MAX_ITER`.
5. اگر index منفی شد -> `+MAX_ITER`.
6. رنگ palette برگردانده می‌شود.

18) `init_iter`

1. اگر set = `JULIA`:
   - `z = pixel` (`z[0]=cx`, `z[1]=cy`)
   - `c = julia parameter` (با overwrite روی `cx/cy`)
2. در غیر این‌صورت (`MANDELBROT`, `BURNING_SHIP`):
   - `z = 0`
   - `c = pixel` همان قبلی.

19) `get_point_iter`

1. `init_iter`.
2. `i=0`.
3. while تا وقتی:
   - `i < MAX_ITER`
   - `|z|^2 < 4`
4. اگر `BURNING_SHIP`:
   - قبل فرمول، `z` قدرمطلق می‌شود.
5. فرمول iteration:
   - `tmp = x^2 - y^2 + cx`
   - `y = 2xy + cy`
   - `x = tmp`
6. `i++`.
7. `return i`.

20) `render`

1. scaleها (`sx`, `sy`) از zoom حساب می‌شوند.
2. `start_x/start_y` برای گوشه بالا-چپ plane.
3. loop روی `y` `0` از `..HEIGHT-1`.
4. pointer سطر با `addr + y * line_length`.
5. loop روی `x` `0` از `..WIDTH-1`.

6. map pixel -> `cx/cy`.
7. `iter = get_point_iter(...)`.
8. color = `get_palette_color(iter, color_shift)`.
9. نوشتن رنگ در buffer.
10. بعد از پایان loopها: `mlx_put_image_to_window`.

44.5) مثال پایان‌به‌پایان (همه لایه‌ها با هم)

سناریو: کاربر wheel up می‌زند.

1. X11 event تولید می‌کند.
2. MLX در `mlx_loop` event را می‌گیرد.
3. `mouse_hook(button=4,x,y,&data)` صدا می‌شود.
4. `zoom_at_point` state را تغییر می‌دهد.
5. `render` دوباره کل frame را می‌سازد.
6. `mlx_put_image_to_window` خروجی جدید را نشان می‌دهد.

این یعنی interaction کامل:
`Event -> Update State -> Recompute Pixels -> Display`.